This illustrates that in addition to weighting the updates on class k by the prediction error $\hat{p}(k|x) - p(k|x)$, the updates are also projected on the orthogonal of the Wx direction. This means that an update on the direction $w_U(k)x^\top$ will occur only to the extent that Wx is not already aligned with $w_U(k)$. Thus, if an association is "stored" once, so that $Wx \approx w_U(y)$, layer-norm will avoid further updating W in that direction, hopefully avoiding norms that grow too much, and also encouraging frequent and infrequent tokens to be weighted similarly in the final memory (see also [8] for more discussion on this).

Note that at random initialization Wx is nearly orthogonal to any $w_U(k)$, so that layer-norm only starts playing a significant role later in training, and does not affect our theoretical analysis based on single gradient steps.

MLP blocks. If we denote by $(u_i)_i$ and $(v_i)_i$ collections of near-orthonormal input and output embeddings, an MLP block may encode associations $(i, j) \in \mathcal{M}$ as follows:

$$F(x) = \sum_{(i,j)\in\mathcal{M}} v_j \sigma(u_i^\top x - b),$$

where σ is a non-linear activation and b a bias term. Then, if one assumes that $u_i^\top u_j \leq b$ for $i \neq j$ and $\sigma(t) = 0$ for $t < 0$, then this can help filter out noise that arises from near-orthogonality, and may then lead to additional storage capacity, at the cost of additional computation (see, *e.g.*, [8, 31]).

An additional benefit of MLP layers discussed in Section 4 is that they may encode many-to-many associations, which is useful when multiple embeddings are in the residual stream and need to be considered jointly (*e.g.*, a subject and a relation in a factual recall task [36]). This may be achieved, for instance, by considering embeddings $u_\mathcal{I} = \frac{1}{\sqrt{|\mathcal{I}|}} \sum_{i\in\mathcal{I}} u_i$, where $\mathcal{I}$ are sets of bounded size (*e.g.*, as obtained using layer-norm over a residual stream). Then, assuming the u_i are nearly-orthonormal, we have $u_\mathcal{I}^\top u_\mathcal{I} \approx 1$ while $u_\mathcal{I}^\top u_{\mathcal{I}'} \lesssim 1 - \delta$ if $\mathcal{I} = \mathcal{I}'$, for some δ that depends on the maximal cardinality of these sets. Although $1 - \delta$ is no-longer vanishingly small, defining

$$F(x) = \sum_{(\mathcal{I},\mathcal{J})\in\mathcal{M}} v_\mathcal{J} \sigma(u_\mathcal{I}^\top x - b),$$

the non-linearity may still succeed at filtering out any $\mathcal{I}$ that does not correspond to the query set in x. We leave the question of how such non-linear associative memories may arise from training dynamics to future work.

Multiple heads and layers. We remark that our view of weights as associative memories applies to any parameter other than embedding/unembedding layers, and thus naturally extends to multiple heads (using the low-rank factorizations described above) and multi-layer models.

It is important to note, however, that the redundancy introduced by having more heads and layers makes it more challenging to identify which layer/head/weight will learn certain associations (see, *e.g.*, Figure 11 in Appendix E). This is in contrast to our simplified architecture of Section 4.2, where we may identify the role of each matrix (up to some possible redundancy when using the feed-forward layer W_F). In practice, mechanisms may appear in different heads/layers across different training runs, which makes interpretability more challenging, and typically requires some causal identification techniques, such as mediation analysis [36, 56].

E Experiment Details and Additional Experiments

In this section, we present additional details on the experiments, as well as additional results.

Computing setup. We use Pytorch and each run uses a single GPU, along with 60 CPU cores for real-time data generation. We will make our code available upon publication.

Hyperparameters. We now provide the hyperparameters used in each figure. The SGD step-size is denoted η. We fix the momentum parameter to 0.9 and the weight decay parameter to 10^{-4}. U denotes the uniform distribution over $[N]$.

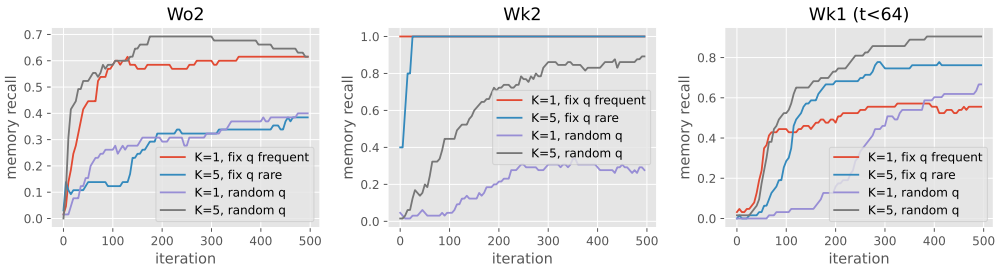

Figure 5: Memory recall probes for the setting of Figure 4(left).

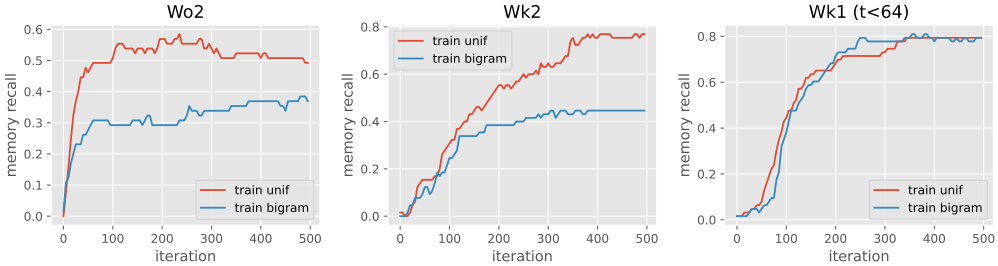

Figure 6: Memory recall probes for the setting of Figure 4(center).

- Figure 2: $K = 3$, $\pi_q = \pi_u$ (random triggers) or Q is the K most likely elements of π_u, $\pi_o = U$, $d = 128$, $d_{hidden} = 4 \times 128$ (hidden dimension of the feed-forward MLPs), $\eta = 0.2$.
- Figure 3: $K = 5$, $\pi_q = \pi_u$ (random triggers), $\pi_o = U$, $d = 128$, $\eta = 0.2$.
- Figure 4(left) and Figure 5: $\pi_o = U$, $d = 128$, $\eta = 1$. For random triggers we use $\pi_q = \pi_u$. For $K = 1$ with fixed frequent trigger, the only trigger is the most probable token according to π_u, while for $K = 5$ with fixed rare triggers, the five triggers are the 6-th to 10-th most probable tokens according to π_u.
- Figure 4(center): $K = 3$, $\pi_q = \pi_u$ (random triggers), $\pi_o = U$ or $\pi_o = \pi_b$ (conditioned on the trigger), $d = 128$, $\eta = 1$.
- Figure 4(right): $K = 3$, $\pi_q = \pi_u$ (random triggers), $\pi_o = U$, $d = 128$, $\eta = 1$.

Memory recall probes and data-distributional properties. Figure 5 and Figure 6 show the evolution of the different memory probes for the settings considered in Figure 4(left,center). Figure 5 highlights that associative memories for the induction head are slower to learn when using few triggers (small K), rare fixed triggers, or random triggers (note that the probe for W_K^2 with fixed triggers only shows recall accuracy on the set of triggers Q, which is an easier task). Figure 6

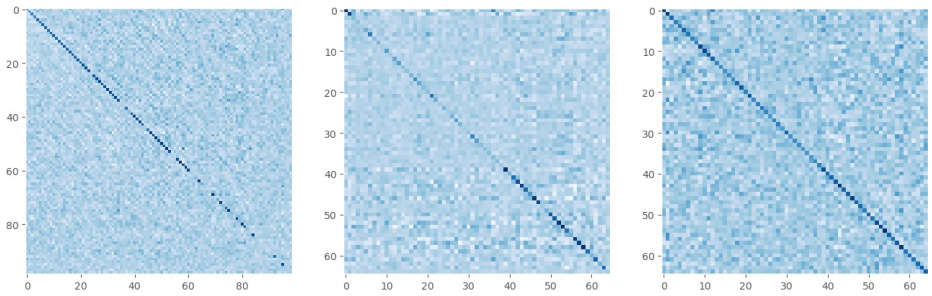

Figure 7: Visualization of the weights W_K^1 (left), W_K^2 (center), and W_O^2 (right) after training with random triggers, $K = 3$, $\pi_q = \pi_u$, $\pi_o = U$. For each of these weight matrices W, if we write the corresponding target memory in (7) as $W_* = \sum_i v_i u_i^\top$ with appropriate embeddings $(u_i)_i$ and $(v_i)_i$ (for instance $u_t = p_{t-1}$ and $v_t = p_t$ for W_K^1 on the left), and we show all values $v_j^\top W u_i$.

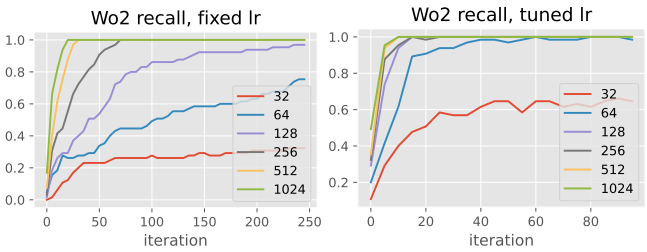

Figure 8: Effect of dimension on learning W_O^2 alone, with fixed or tuned learning rate.

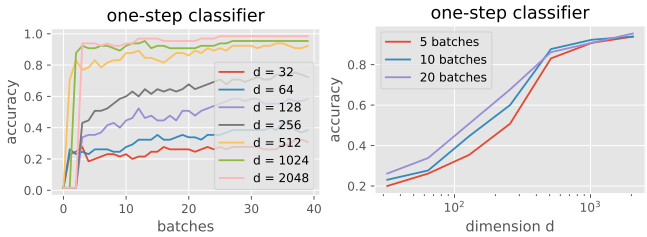

Figure 9: Accuracy of one-step estimate of W_O^2 with varying dimension and number of batches used for computing expectations. Each batch consists of 32 sequences of 256 tokens for a total of 8 192 tokens, with $K = 5$ random triggers and uniform outputs.

shows that using uniform output tokens can lead to better fitting of W_O^2 and W_K^2 compared to using output tokens sampled from bigrams. In addition to the increased diversity when using uniform outputs, this may also be due to the fact that bigram outputs are already well predicted using global statistics with just the feed-forward layer, hence the gradient signal on such well-predicted tokens may not propagate through the induction head mechanism. In contrast, the recall accuracy for W_K^1 is comparable for both settings, since the previous token head is useful at all positions regardless of the output token distribution.

Visualizing memories. Figure 7 shows visualizations of the associative memory behaviors after training. We see that diagonal elements dominate in the plots, which corresponds to correct associations lead to high 'memory recall'. Nonetheless, we see that some of the diagonal elements are weaker than others, particularly for late positions in W_K^1, and for some of the trigger tokens in W_K^2, while the diagonal for W_O^2 seems to be roughly uniform. We note that characters corresponding to capital letters have token index 13 to 38, while lowercase letters have index 39 to 64. The association patterns found in W_K^2 then seem related to frequencies of appearance of triggers, whereby capital letters appear less frequently in the data, and are also less frequently chosen as triggers, compared to lowercase letters. Similarly, since the first occurrence of triggers is typically early in a sequence, it is natural that W_K^1 learns stronger associations at earlier positions. In contrast, diagonal elements for W_O^2 are nearly uniform, which agrees with the fact that output tokens are sampled uniformly in this setup. We refer to the follow-up work [8] for an analysis of how data frequencies affect association strength in such associative memories.

Effect of dimension. Recall that our study of associative memories with random embeddings requires large dimension d in order to ensure near-orthogonality, and thus store input-output pairs more effectively. In Figure 8, we evaluate the recall accuracy for W_O^2 for varying dimension, when training it by itself, and only on the output tokens (as in Figure 3). We see that higher dimension leads to faster learning of the memory, in particular $d = 128$ seems sufficient for fast learning after just a few iterations with a tuned learning rate. If the learning rate isn't tuned, we notice that there is a further slowdown for low dimension, is likely due to issues with the fact that our experiments use the standard parameterization of neural networks at initialization, rather than maximal update parameterizations [60]. Note that learning W_O^2 alone is a convex optimization problem, and we hypothesize that higher dimension makes the problem better conditioned, and hence easier to learn. In Figure 9, we show "one-step" recall accuracies for classifying output tokens from the average